Day 6 (Evening Session)

JavaScript Fundamentals: ECMAScript, Node.js, Modules, fs and http

Lecture notes - Instructor: Abhishek Purohit - Date: 26 Oct 2026 - Notes for: Sumeet Sharma

#	Topic covered in this lecture
1	ECMAScript (ES): the blueprint for JavaScript, history (LiveScript vs JScript), CommonJS vs ES
2	Node.js: what it is and is not, why it exists, V8 engine, versions, installation, running .js files
3	Modules: concept, predefined vs user-defined, the require() function
4	fs module: writeFile / writeFileSync / readFile / readFileSync, async vs sync, Buffer
5	http module: building a server, createServer, callback, listen, IP address, port, socket
6	User-defined (custom) modules: creating, exporting and requiring them
7	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

How to read these notes
Blue boxes hold the instructor's explanation (converted from spoken to written English). Yellow boxes are additional explanations added by me to help you; they were **not** said in the lecture. Red boxes list common mistakes. Every program from your class notes is included with explanation and expected output. This lecture did not cover the DOM, events or forms; those topics are therefore not part of these notes.

1. ECMAScript (ES)

1.1 What is ECMAScript?

Instructor's explanation
ECMAScript is **not a language**. Practically, you always write your code in JavaScript, but there is an organisation called **ECMA** that controls the entire behaviour of JavaScript. In simple words, **ECMAScript is just a blueprint for JavaScript**. It defines the rules, regulations and specifications, and decides which features JavaScript should have. JavaScript is the language; its rules are defined by ECMAScript; JavaScript then implements all those functionalities.

Example given in class: suppose ECMA decides there should be asynchronous programming. It only defines the *concept* (async programming, callback functions). JavaScript then implements it, and you write code using that new feature. You still write JavaScript, but behind the scenes ECMAScript defines how JavaScript should behave.

1.2 Why was ECMAScript needed? (History)

To understand why a blueprint was needed, we go back to the early 1990s.

Year / Period	Event	Who
Early 1990s	JavaScript was introduced under the name **LiveScript** to make web pages interactive.	Netscape (a browser company)
After its success	Microsoft saw LiveScript's popularity and created its own language for the same purpose, named **JScript**.	Microsoft
The conflict	Netscape's browser supported only LiveScript. Microsoft's browser supported only JScript.	Both companies
Around 1996-97 (instructor's recollection)	ECMA stepped in and defined common specifications. Every browser had to support **ECMAScript**. The language was given the name **JavaScript**.	ECMA

Instructor's explanation
Who suffered from this fight, the user or the developer? It is always the **developer**. A user simply reports a bug if a feature does not work, but a developer now had to write **different code for different browsers**: JScript code for Microsoft's browser and LiveScript code for Netscape's browser. For about five to six years (roughly the mid-1990s) this was a nightmare for developers. To end this fight, ECMA came in the middle and, in the instructor's dramatic retelling, told both companies to put their code aside: from now on one specification decides the rules, the syntax and the blueprint. After that, every browser had to support ECMAScript, browser-specific languages faded away, and JavaScript gained a lot of popularity.

Additional note (not from the lecture)
The instructor said he was not a developer at that time and was sharing what he has read in history, so treat the exact years as approximate.

1.3 Who introduces new features today?

Whenever new features arrive, it is **ECMAScript** that introduces them. JavaScript only implements them. Defining things is in the hands of ECMAScript; JavaScript simply follows. Every year a new ECMAScript version is released and JavaScript provides the implementation. The instructor checked during class that **ES2026** is the latest version and **ES2027** is upcoming.

Features mentioned in class that come from ECMAScript: improved string literals, arrow functions, JSON-style objects, Promises, and object-oriented support such as the `class` syntax and `inheritance`.

1.4 ES6 and classes

ECMAScript releases are numbered in increments: ES6, ES7 and so on. The `class` syntax and the inheritance syntax were introduced in **ES6**. Before ES6, JavaScript had **no native feature to create a class**. You had to simulate class-like behaviour through **functions and prototypes**. The instructor's remark: "imagine what kind of workaround that was, implementing a class through a function."

1.5 CommonJS vs ECMAScript

Instructor's explanation
There are two conventions (standards) for writing JavaScript. The older one is **CommonJS** (the regular JavaScript code before ECMAScript, where class-like behaviour used prototypes) and the newer one is **ECMAScript**. In modern applications we should always go with ECMAScript standards. For example, arrow functions and the `class` keyword are conventions of ECMAScript, not CommonJS.

Point	CommonJS (older convention)	ECMAScript (modern convention)
Classes	No `class` keyword; simulate with functions + prototypes	Direct `class` keyword support
Arrow functions	Not part of the convention	Part of ECMAScript
Use today	Older style	Preferred in modern applications
In this lecture	The module code written later (`require`, `exports`) is the CommonJS style	Will be covered in the next session

Additional note (not from the lecture)
Strictly speaking, CommonJS is the module system that Node.js originally used (`require()` and `exports`), while ECMAScript modules use `import` / `export`. This matches the instructor's closing remark: the syntax used in this lecture is the CommonJS convention, and the next session will use the latest ECMAScript syntax.

2. Node.js

2.1 What is Node.js?

In class, students answered: "a runtime environment" and "it is used to run JavaScript code on a server". Both are correct.

> **Instructor's explanation**
> Beginners often think Node.js is another language, library or framework that they must learn. It is **none of these**. Node.js is simply an **interpreter**, and the job of an interpreter is to run your code, nothing more. Node.js is a **runtime environment for JavaScript**: a piece of software that executes your JavaScript code.

Node.js IS	Node.js is NOT
A runtime environment for JavaScript	A programming language
An interpreter that runs your JavaScript code	A library
Free, open-source, cross-platform software (Windows, macOS, Linux)	A framework

2.2 Analogy with Java

You already know Java: the JVM/JRE runs your Java code. In the same way, **Node.js is to JavaScript what the JVM/JRE is to Java.** Just as you create a *.c*, *.cpp* or *.java* file and run it independently, you can now create a *.js* file and run it with Node.js.

2.3 Why do we need Node.js?

Phase 1 - The initial approach: browser only.

> **Instructor's explanation**
> JavaScript code did run without Node.js, but only inside a browser. You had to put the code inside a `<script>` tag in an *.html* file, and the browser's JS engine ran it. There was no other component that could run JavaScript. A server does not run inside a browser; it must run as an independent process, like a C, C++ or Java program.

Phase 2 - The problem: JavaScript depended on the browser, so you could not write a server or backend in JavaScript.

Phase 3 - The improvement: some people took the **JS engine out of the browser**, modified it and added more features, and named the result **Node.js**. Now a *.js* file can run directly on your machine with no browser and no HTML file. With Node.js you can create servers and a complete backend. A browser is still needed later for the front end or for sending requests, but **no longer needed just to run JavaScript**.

2.4 The V8 engine

> **Instructor's explanation**
> Node.js uses the **same JavaScript engine that Google Chrome uses, called the V8 engine**. Node.js is implemented on top of V8. Do not think of Node.js as an official runtime released by JavaScript or by Chrome; it is a separate project.

2.5 Which Node.js version should you use?

> **Instructor's explanation**
> In industry, many projects have migrated to **version 22**; version 18 was deprecated long ago. You can use version 22 or **version 24**. Avoid the very latest 'Current' release because it can contain bugs and vulnerabilities (the instructor mentioned a recent PHP vulnerability and an earlier security issue in the Axios library as examples of why). Use the stable version just before the latest.

2.6 Installing Node.js and verifying it

1. Open **nodejs.org**, go to the Download section and download the installer for version 22 or 24 (as shown in class, version 24).
2. If an old version exists, uninstall it first (the instructor uninstalled his version 22 before installing 24).
3. Run the installer like any normal software: Next, accept the terms and conditions, Next, Next, Install.
4. Open a terminal / command prompt and type `node` and press Enter. A welcome message with the Node.js version appears, which means it is installed.
5. To check the version only, type `node --version` or the short form `node -v` (double hyphen for the full word, single hyphen for the letter).

Command	Purpose
`node`	Starts the interactive Node.js console (welcome message appears)
`node --version`	Prints the installed version
`node -v`	Short form of the same command
`node filename.js`	Runs a JavaScript file with Node.js

Instructor's explanation
If you see the error *'node' is not recognized as an internal or external command*, the **PATH is not set**, as with the JDK for Java. On Windows: Settings > System > About > Advanced system settings > Environment Variables > Path, then add the Node.js path. The installer normally adds it by itself; set it manually only if that did not happen.

2.7 Running your first JavaScript file on Node.js

Purpose: run JavaScript without HTML or a browser. Create a file named `demo.js` and open it in VS Code. Do **not** write a script tag; write JavaScript directly.

```
console.log("hello world");
```

1. Open a terminal in the folder that contains `demo.js`. In Windows 10/11, right-click the folder and choose **Open in Terminal**; otherwise use `cd folder-name`.
2. There is no compilation step for JavaScript. Run: `node demo.js`

```
Output:
hello world
```

Why this output appears: `console.log` now prints to the **Node.js console (terminal)**, not the browser console.

Additional note (not from the lecture)
The instructor also installed a Node-specific extension in VS Code (a 'Node Essentials' type extension) to get suggestions such as `fs.writeFile` while typing. This is optional.

3. Modules

3.1 What is a module?

Instructor's explanation
A module is a **collection of functions, classes, properties, constants and predefined code** that you can reuse in your code. What a **header file** is in C/C++ and what a **package** is in Java, the **module** is in JavaScript. It is like a bundle or package. Example: for file operations you get a module with predefined file functions; to implement a server you get the functions inside another module.

Everything in JavaScript is an object, so even a module, once loaded, is treated like an object.

3.2 Types of modules

Type	Meaning	Examples
Predefined (built-in)	Already available in Node.js by default	fs, http, path, os
User-defined (custom)	Created by developers for their own projects	Your own mymodule.js

3.3 Some predefined modules

Node.js has a large list of predefined modules. The lecture focuses on frequently used ones (fs and http) but the instructor showed the list from the documentation. Read the Node.js documentation for the rest.

Module	Purpose
`fs`	File system: write, read, delete, rename files
`fs/promises`	Promise version of fs
`http`	Create a server, accept requests, send responses
`path`	Work with file and folder paths
`os`	Information about the operating system
crypto, events, stream, buffer, url, child_process, util	Other predefined modules (listed in class, not explained)

3.4 How to use a module: the require() function

In Java you first *import* a package and then use its classes; in C/C++ you first write *#include* and then use its functions. In Node.js you first **require** the module and then call its functions. `require()` does the job of `#include` (C) and the *import* statement (Java).

```
const fs = require("fs");      // load the fs module
const http = require("http");  // load the http module
const path = require("path");  // load the path module
```

1. Pass the **name of the module** (a string) to `require()`.
2. `require()` attaches (includes) the module to your program and **returns a reference** to it.
3. Store that reference in a variable. It is recommended to give the variable the **same name as the module** for readability (you could use xyz or abc, but readability suffers).
4. Call the module's functions on that reference: `reference.functionName()`. Without the reference you cannot call the module's functions.

Generic example from class: a module named `mymodule` that has two functions `f1` and `f2`.

```
const myModule = require("mymodule");  // require + store the reference

myModule.f1();   // call function f1 of the module
myModule.f2();   // call function f2 of the module
```

4. The fs (File System) Module

fs stands for **file system**. It gives you functions to write data to a file, read data from a file, delete a file, rename a file, and more. A student asked whether fs is synchronous. The instructor's answer: **both**; for most operations you get both an asynchronous and a synchronous function.

Operation	Asynchronous (callback)	Synchronous
Write data into a file	`fs.writeFile()`	`fs.writeFileSync()`
Read data from a file	`fs.readFile()`	`fs.readFileSync()`

Functions ending with `Sync` block the rest of the program until they finish; the others do not. To see the exact arguments and their order, read the Node.js documentation (nodejs.org, fs section) or W3Schools / GeeksforGeeks.

For simplicity the lecture works only with **.txt files** (no special formatting). Other types such as PDF or images are also possible.

4.1 Writing a file asynchronously: fs.writeFile()

Purpose: create a text file and write data into it without blocking the rest of the code.

Behaviour: if the file already exists, its content is **overwritten**; if it does not exist, it is **created** in the same folder where the program is run, then the data is written.

Incorrect approach first: no callback

```
const fs = require("fs");

console.log("Before writing into file");
fs.writeFile("my-file.txt", "this is dummy file");   // no callback
console.log("after writing file ");
```

Common mistake
Node.js throws an error saying the **callback argument must be of type function** (seen in class). The asynchronous write function must be given a callback, which Node calls **after the write operation completes**.

Corrected approach

```
const fs = require("fs");

console.log("Before writing into file");
fs.writeFile("my-file.txt", "this is dummy file", () => {
    console.log("write op completed");
});
console.log("after writing file ");
```

Line-by-line explanation

- `fs.writeFile("my-file.txt", ...)`: first argument is the **file name** to write into.
- Second argument `"this is dummy file"` is the **data** (content) to write.
- Third argument is the **callback function**. It runs only once the write has finished.
- The two `console.log` lines before and after are placed to observe the order of execution.

```
Output:
Before writing into file
after writing file
write op completed
```

A new file **my-file.txt** is created containing: *this is dummy file*.

> **Instructor's explanation**
> Look at the sequence of output. 'After writing file' appears **before** 'write op completed'. So the function runs **asynchronously**: while data is being written, the rest of the code is not blocked and continues to execute. As soon as the write operation completes, Node calls the callback.

4.2 Writing a file synchronously: fs.writeFileSync()

To do the same job synchronously use `writeFileSync`. A callback is **not needed**, because callbacks are only required for asynchronous functions.

```
const fs = require("fs");

console.log("Before writing into file");
fs.writeFileSync("new-file.txt", "this is new dummy file");
console.log("write op completed");
console.log("after writing file ");

Output:
Before writing into file
write op completed
after writing file
```

Why this order: the program waits at `writeFileSync` until the data is completely written; only then does the rest of the code run. The order is guaranteed. The result is the same as before: a file **new-file.txt** with the content *this is new dummy file* is created (or overwritten if it exists).

4.3 Synchronous vs asynchronous

Point	Asynchronous (writeFile / readFile)	Synchronous (writeFileSync / readFileSync)
Rest of the code	Not blocked; continues executing	Must wait until the operation finishes
Callback	Required	Not required
Result delivery	Passed to the callback	Returned by the function
Order of output	Not sequential	Guaranteed sequential
Choice	Depends on your requirement; both reach the same final result	

4.4 Reading a file asynchronously: fs.readFile()

Purpose: read data from an existing file. Pass the file name and a callback. When reading completes, Node calls the callback and gives you the data.

Version 1: basic read

```
const fs = require("fs");

fs.readFile("my-file.txt", (error, data) => {
    console.log(data.toString());
});

console.log("after reading file");
```

Explanation

- The callback receives **two arguments**: the first is the **error**, the second is the **data**. (In class, logging the first argument printed *null*, which is how the instructor discovered there are two arguments.)
- If the read succeeds, **error is null** and data contains the file content. The read may fail because the file does not exist or read permission is missing; then an error object is provided.
- `data.toString()` converts the raw bytes into a readable string (explained below).
- While the file is being read the CPU is not idle; it continues with the rest of the code, so 'after reading file' prints first.

```
Output:
after reading file
this is dummy file
```

> **Instructor's explanation**
> The data does not arrive directly as text. You get it in the format of a **buffer**, that is, **bytes**. Compare Java file handling: there are two types of streams, the byte stream and the character stream. With a byte stream you always get bytes. The same thing happens here. Whenever you read any file you get a buffer (bytes), and if you know the file is a .txt file with text, you must convert it to a string by calling **toString()**.

Without `toString()` the output would look like this (supplementary illustration):

```
fs.readFile("my-file.txt", (error, data) => {
    console.log(data);          // no toString()
});

Output:
<Buffer 74 68 69 73 20 69 73 20 64 75 6d 6d 79 20 66 69 6c 65>
```

Each pair of hexadecimal digits is one character (for example 74 is the letter 't').

Version 2: improved with error handling

```
const fs = require("fs");

fs.readFile("my-file.txt", (error, data) => {
    if (error) {
        console.log(error);              // read failed: show the error
    } else {
        console.log(data.toString());    // read succeeded: show the content
    }
});

console.log("after reading file");
```

Logic: if `error` has a value, log the error; otherwise we have the data, so log it as a string. The output is the same as Version 1 when the file exists. The proof of asynchronous behaviour is that 'after reading file' is printed first.

> **Additional note (not from the lecture)**
> If the file name is wrong, Version 1 would crash because `data` would be undefined. Version 2 avoids this (this callback style, with the error as the first argument, is often called an error-first callback).

4.5 Reading a file synchronously: fs.readFileSync()

```
const fs = require("fs");

const data = fs.readFileSync("my-file.txt");   // returns the data directly
```

```
console.log(data.toString());

console.log("after reading file");
```

Explanation: no callback is needed; the function **returns** the file data (as bytes), so `toString()` is again used to convert it to a string.

```
Output:
this is dummy file
after reading file
```

Why: the program waits until the file is read, so the file content is printed first and then the next line. Choice is yours: synchronous (no callback, the function returns the data) or asynchronous (data comes inside the callback).

4.5.1 Other fs functions

There are further functions in the fs module, for example to **remove** a file or **rename** a file. Look up the full list in the fs documentation.

5. The http Module and Creating a Server

5.1 What does the http module do?

Instructor's explanation
By its name, the HTTP module provides functions to **implement a server**: a program that can accept a request, process the request and generate the response. A server runs **continuously** (for an indefinite time) and **continuously listens** for incoming requests. A client (a browser or anything else) sends a request; as soon as it arrives the server accepts it, processes it, and at last generates a response that goes back to the client.

5.2 Step 1: create a server

First require the module, then call `createServer()` on its reference. It creates a server and **returns a reference to that server**, which we store in a variable (name of your choice, for example *server*).

```
const http = require("http");

const server = http.createServer((request, response) => {
    response.write("Hello World");
    response.end();
});
```

The callback function and its two arguments

Instructor's explanation
Inside `createServer()` you pass a **callback function with two arguments**. You can name them anything (x, y or p, q), but the **first argument always represents the incoming request object** and the **second always represents the outgoing response object**. Everything is an object in JavaScript: the server, the request the browser sent, and the response are all objects. Use the request object to read data from the request and the response object to send a response.

Instructor's explanation
As developers we **never call this callback ourselves**. It is called automatically **whenever a client sends a request to the server**. When you run the JS file (`node demo.js`) the server is only created; the callback does **not** run at that moment. This callback is the most important function: all the logic that accepts the request, processes it and generates the response goes inside it. For example, if a client asks for an addition, the addition logic and the result in the response go here.

Sending the response: write() and end()

- `response.write("Hello World")` only **specifies what goes in the response**. The response has not actually gone yet.
- `response.end()` tells Node that processing is complete, the cycle ends, and the response is sent to the client.

The instructor first typed 'response.send' and then corrected it to `response.write` after recalling the exact method.

5.3 Problem: the server is created but not started

If you run the above code with `node demo.js`, the program simply finishes and exits. Nothing is listening.

Instructor's explanation
With the `createServer()` function **only a server will be created; it will not get started**. Creating a server is not enough; you have to start it as well. The proof is that as soon as you ran the code, nothing continued to happen. Starting a server means it must **continuously listen for requests**. Before starting, we need to understand the IP address and the port number.

5.4 IP address, port number and socket

IP address

A **unique identity given to a system on a network** (the 'on a network' part is important; a system not connected to a network has no IP, and the IP can change from network to network). Your server has an IP address too. In class the server runs on your own computer, so we use the fixed local address:

Local machine address	Value
IP address	`127.0.0.1`
Domain name	`localhost`

Port number

> **Instructor's explanation**
> Many processes can run on one server: an e-commerce application, a social media application, a database and so on, all on the same IP address. The IP address alone cannot identify which process a request is meant for. Think of a **railway station**: the station address is the IP, but to catch the right train you also need the **platform number**. That platform number is the **port number**. A port number **uniquely identifies a process running on a server**, and the developer decides it.

Application	Port (example)	Address typed in the browser
E-commerce application	7654	`127.0.0.1:7654`
Social media application	8100	`127.0.0.1:8100`
MySQL database (default)	3306	`127.0.0.1:3306`

Socket = **IP address + port number** combined (written with a colon, for example 127.0.0.1:6500).

Which port numbers should you avoid?

> **Instructor's explanation**
> Avoid port numbers that are probably already taken, such as **3306, 8080, 8090, 3000, 4200, 8000** (default ports of MySQL, Java servers, React, Angular and so on; they can be customised, but these are common defaults). If the port is already taken, you will get an error when starting the server. In this lecture the port used is **6500**.

5.5 Step 2: start the server with listen()

Call `listen(portNumber)` on the server reference. It **starts the server** and makes it continuously listen for requests on that port.

```
const http = require("http");

const server = http.createServer((request, response) => {
    response.write("Hello World");   // what to send
    response.end();                  // finish and send the response
});

// createServer() only creates the server; the callback above does NOT run now.
// The callback runs only when a client sends a request to this server.

server.listen(6500);  // starts the server and listens continuously on port 6500
```

Execution flow

1. Run `node demo.js`. The server is created and `listen(6500)` starts it. The program does **not end**; it keeps running.
2. Open a browser and type `127.0.0.1:6500` (or localhost:6500) and press Enter.
3. The request reaches the server, so the callback executes. It writes 'Hello World' and ends the response.

4 The browser displays the response.

5 To stop the server, press **Ctrl + C** in the terminal (the instructor did this before running the next program).

```
Output:
Browser page shows:  Hello World
```

Additional note (not from the lecture)
If you change the server code, stop it with Ctrl + C and run it again. If you start two servers on the same port, Node reports an 'address already in use' type error. The instructor said more things can be done with the server in later sessions.

Step	Code	What happens
1	`require("http")`	Loads the http module
2	`http.createServer(callback)`	Creates a server only (does not start it)
3	`(request, response) => {...}`	Runs on every client request; processes it and sends the response
4	`response.write()`	Defines the response content
5	`response.end()`	Finishes and sends the response
6	`server.listen(6500)`	Starts the server and listens on port 6500

6. User-Defined (Custom) Modules

Instructor's explanation
To create your own module, you just create a **.js file** and put various functions inside it. That is all. Then you require that file in another JS file and call its functions.

Purpose: create a module with two functions and use it from *demo.js*. Both files must be in the same folder.

Phase 1: create the module file (mymodule.js), initial attempt

```
// mymodule.js
function sum(a, b) {
    return a + b;
}

function product(a, b) {
    return a * b;
}
```

Phase 2: use it in demo.js and see the problems

In the instructor's demonstration, the http server code in demo.js was commented out first. Note: the earlier http server must be stopped (Ctrl + C) before running again.

```
// demo.js  (attempt 1: wrong path)
const myModule = require("mymodule.js");
console.log(myModule.sum(2, 5));
```

Common mistake
Error: Cannot find module. For your own module you must give a **path**. A name with no path is treated as a predefined module. Use `./` (current folder) before the file name.

```
// demo.js  (attempt 2: path is correct, but the function is not exported)
const myModule = require("./mymodule.js");
console.log(myModule.sum(2, 5));
```

Common mistake
Error: myModule.sum is not a function. The module is now found, but a function defined in one file cannot be called from another file unless the first file **exports** it. Only the file where the function is defined can call it directly; to expose it to other files you must do the export and import (require) business.

Phase 3: export the functions

The instructor used the anonymous function syntax with `exports.name = function(){}`. Each function is exported with the property name you give it.

```
// mymodule.js
exports.sum = function (a, b) {
    return a + b;
};

exports.product = function (a, b) {
    return a * b;
};

// demo.js
const myModule = require("./mymodule.js");

console.log(myModule.sum(2, 5));
console.log(myModule.product(3, 8));

Output:
7
24
```

Explanation

- `exports.sum` exports the first function under the name **sum**; `exports.product` exports the second under the name **product**.
- `require("./mymodule.js")`: the path starts with `./` because the file is in the current folder (the two files are at the same level).
- `myModule.sum(2, 5)` calls the exported function; 2 + 5 = 7. `myModule.product(3, 8)` gives 3 x 8 = 24.
- The rule: **module name . function name** to call a function of a module.

Additional note (not from the lecture)
The instructor typed the full file name with `.js`; Node also allows `require("./mymodule")` without the extension.

Instructor's explanation
The syntax we used here is the **CommonJS convention**, not the latest ECMAScript syntax. In the next session we will discuss the ECMAScript syntax (such as class and the improved module syntax) further.

7. Revision Material

7.1 Summary of all major topics

- **ECMAScript** = blueprint/specification for JavaScript, maintained by ECMA. JavaScript implements it. New versions every year (ES6 added classes; ES2026 latest, ES2027 upcoming).
- **History:** early 1990s LiveScript (Netscape) vs JScript (Microsoft) created chaos for developers; ECMA standardised the language.
- **CommonJS vs ES:** older style (prototypes, require/exports) vs modern style (class keyword, arrow functions).
- **Node.js** = interpreter / runtime environment for JavaScript, built on Chrome's V8 engine. Not a language, library or framework. Runs .js files without a browser.
- Use a stable Node version (22 or 24); avoid the very latest current release.
- **Module** = collection of functions, classes and properties (like C headers / Java packages). Predefined (fs, http, os, path) or user-defined.
- **require(name)** loads a module and returns a reference; call functions through that reference.
- **fs**: writeFile / writeFileSync / readFile / readFileSync. Async needs a callback; sync does not. Read data arrives as a Buffer; use toString().
- **http**: createServer(callback) creates; listen(port) starts. Callback runs only on a client request; request first argument, response second.
- **IP + port = socket.** localhost = 127.0.0.1. Port identifies the process like a platform number identifies a train.
- **Custom modules**: a .js file; export with exports.name = function; require with a path (./file.js).

7.2 Important definitions and syntax

Term / Syntax	Meaning
ECMAScript	The specification (blueprint) that defines how JavaScript behaves
Node.js	Runtime environment / interpreter that runs JavaScript outside the browser
V8	The JavaScript engine of Chrome, also used by Node.js
Module	A collection of reusable functions, classes and properties
`const x = require("name")`	Load a module and keep its reference in x
`fs.writeFile(file, data, callback)`	Write asynchronously
`fs.writeFileSync(file, data)`	Write synchronously
`fs.readFile(file, (error, data) => {})`	Read asynchronously
`fs.readFileSync(file)`	Read synchronously; returns a Buffer
`http.createServer((request, response) => {})`	Create a server (does not start it)
`server.listen(port)`	Start the server and listen on the port
`exports.name = function(){}`	Export a function from a custom module
IP address	Unique identity of a system on a network (local: 127.0.0.1)

Term / Syntax	Meaning
Port number	Unique identity of a process running on a server
Socket	IP address + port number

7.3 Key differences between related concepts

Concept A	Concept B	Key difference
JavaScript	ECMAScript	JavaScript is the language; ECMAScript is the specification it follows
LiveScript (Netscape)	JScript (Microsoft)	Two browser-specific languages with the same purpose; caused incompatibility
CommonJS	ECMAScript	Old convention (prototypes) vs modern syntax (class, arrow functions)
Node.js	Library / framework	Node.js only runs JavaScript; it is not a library or framework
Predefined module	User-defined module	Built into Node.js vs created by the developer
writeFile	writeFileSync	Non-blocking with callback vs blocking with no callback
readFile	readFileSync	Data given to callback vs data returned by function
createServer()	listen()	Creates the server vs starts it
IP address	Port number	Identifies the system vs identifies the process on that system

7.4 Common mistakes to avoid

Mistake	Why it fails	Correction
Calling writeFile / readFile without a callback	Error: callback must be a function	Pass a callback, or use the Sync version
Printing data from readFile directly	You get a Buffer (bytes)	Use data.toString()
Assuming async code runs in written order	Async callback runs later	Remember the output order; use Sync if order matters
Writing only createServer() and expecting a running server	Server is created but not started; program exits	Call server.listen(port)
Expecting the createServer callback to run at start-up	It runs only on a client request	Send a request from the browser
Using a port already in use (3306, 8080, 3000...)	Error on starting the server	Choose a free port such as 6500
require("mymodule.js") for your own file	Cannot find module	Use a path: require("./mymodule.js")
Not exporting functions in a custom module	myModule.sum is not a function	Use exports.sum = function...
'node is not recognized' after install	PATH not set	Add the Node.js path to the environment variables
Using a bleeding-edge Node version	Possible bugs and vulnerabilities	Use a stable version (22 or 24)

7.5 Interview questions and answers

Items 1-8 follow the lecture; items marked (extra) go beyond the lecture.

Question	Answer
What is ECMAScript?	A specification / blueprint maintained by ECMA that defines the rules and features of JavaScript.
Why was ECMAScript created?	Netscape's LiveScript and Microsoft's JScript were incompatible and forced developers to write different code per browser. ECMA standardised the language.
What is Node.js?	A runtime environment (interpreter) that runs JavaScript outside the browser, built on the V8 engine. It is not a language, library or framework.
Which engine does Node.js use?	The V8 engine, the same as the Chrome browser.
What is a module?	A collection of reusable functions, classes and properties, similar to a C header file or a Java package.
Types of modules in Node.js?	Predefined (fs, http, os, path) and user-defined (custom).
What does require() do?	Loads a module and returns a reference to it (like #include in C or import in Java).
Difference between writeFile and writeFileSync?	writeFile is asynchronous and needs a callback; writeFileSync blocks until finished and needs no callback.
Why do we call toString() on file data?	Files are read as a Buffer (bytes); toString() converts it into a string.
What does createServer() do and does it start the server?	It creates a server and returns its reference; it does not start it. listen(port) starts it.
When is the createServer callback executed?	Only when a client sends a request to the server.
What are request and response in the callback?	The first argument is the incoming request object, the second is the outgoing response object.
What is a port and a socket?	Port uniquely identifies a process on a server. Socket = IP address + port.
How do you make functions of a custom module usable elsewhere?	Export them (exports.name = function...) and require the file with its path.
(extra) What is the difference between CommonJS and ES modules?	CommonJS uses require / exports; ES modules use import / export (next session).

7.6 Practice exercises (solutions not included)

Easy

1. Write one sentence explaining why Node.js is not a language, a library or a framework.
2. Create hello.js that prints your name using console.log and run it with Node.js.
3. Run `node -v` and note your installed version.

Medium

1. Using fs.writeFileSync, create notes.txt containing three words, then read it back with fs.readFileSync and print it as a string.
2. Repeat exercise 1 with writeFile and readFile and observe the order of output of the console messages. Explain why it differs.
3. Remove the callback from fs.writeFile and read the error. What does it tell you?
4. Create a custom module with two functions, difference(a, b) and square(a), export them and call them from another file.

Challenging

1. Create an http server on port 7000 that responds with your name. Open it in the browser using localhost:7000.
2. Start your server twice on the same port in two terminals. What happens, and why?
3. Write a readFile program that prints a friendly message if the file does not exist, instead of crashing.
4. In the createServer callback, add one more response.write call before response.end() and observe the browser output.

7.7 Final revision checklist

- [ ] I can explain ECMAScript as a blueprint and why it was needed (LiveScript vs JScript).
- [ ] I know CommonJS (prototypes) vs ES (class) and that this lecture's require/exports style is CommonJS.
- [ ] I can explain what Node.js is and is not, and the Java JVM analogy.
- [ ] I know Node.js uses the V8 engine and which versions to use.
- [ ] I can install Node.js, verify it with node -v and fix a PATH problem.
- [ ] I can run a .js file with node filename.js.
- [ ] I know what a module is, the two types, and how require() works.
- [ ] I can write and read files using both async and sync fs functions and predict the output order.
- [ ] I understand Buffer and why toString() is used.
- [ ] I can create a server with createServer and start it with listen.
- [ ] I know why the createServer callback is not called at start-up.
- [ ] I can explain IP address, port number and socket and avoid common ports.
- [ ] I can create, export and require my own module with the correct ./ path.